SyntheSift

AI-Powered Data Quality Analysis & Remediation Platform

Technical documentation and reproducible acceptance guide

Prepared for software engineering and generative AI project assessment. This document describes the implemented architecture and an evidence-collection protocol; it does not claim deployment or measurement results that have not been independently recorded.

Scope: CSV admission, statistical profiling, deterministic quality checks, Gemini-assisted recommendations, explicit human approval, safe transformations, validation, and export.

Stack: Next.js / React / TypeScript; Django REST Framework; PostgreSQL; Redis + Celery; Gemini API; Docker Compose; pytest, Vitest, Ruff and GitHub Actions.

Document state: implementation description and verification template. Attach current CI screenshots, measured metrics, and a successful live Gemini example before submission.

SECURITY PRINCIPLE

Evidence-grounded suggestions are advisory. Only deterministic, allowlisted and expressly approved operations may create a derived dataset. Original sources remain immutable.

1. Real-world problem and product workflow

Data teams frequently receive spreadsheets with missing entries, repeated records, inconsistent formatting and suspicious numeric values. A common failure is to rely on ad-hoc scripts without preserving provenance or showing what changed. SyntheSift combines measurable profiling, limited generative explanations and a controlled remediation audit.

Phase	Behavior and guarantee
Admit	Validate UTF-8 CSV, structure and upload limits; save immutable source with SHA-256.
Profile	Compute row/column statistics, missingness and rule-based quality findings.
Generate	Send approved aggregate evidence and column labels to Gemini, not original CSV rows.
Review	Show prioritized recommendations with evidence identifiers; do not auto-execute model text.
Remediate	Preview allowlisted transformations and require an explicit owner/editor approval.
Validate	Recalculate derived-dataset statistics and preserve source/output hashes.
Export	Provide derived CSV and PDF audit through authorized, non-cacheable endpoints.

2. Architecture and trust boundaries

- **Frontend:** authenticated Next.js workspace and project UI; viewer permissions are reflected in controls but enforced again on the server.
- **Backend:** Django REST endpoints validate permissions, datasets, profile states and approved remediation operations.
- **State:** PostgreSQL stores metadata, evidence, report status, and audit lineage; original/derived CSVs are separately checksummed.
- **Async:** Celery workers use Redis for admission, profiling and Gemini generation. Provider keys are supplied only to backend/worker environments.
- **External boundary:** Gemini receives constrained aggregate profiling evidence after explicit disclosure consent. Prompt data are treated as untrusted.

3. Four prompt engineering approaches

Technique	Strategy / inspection focus
Zero-shot	Directly request bounded, evidence-linked recommendations from the aggregates.
Few-shot	Use an illustrative example to demonstrate required JSON shape; no example facts carry over.
Structured reasoning	Ask for internal comparison/prioritization and concise final conclusions, not hidden reasoning.
Role-based	Use a cautious data-steward persona to emphasize reversible, reviewable actions.

Supported generation parameters include model, maximum output tokens, Gemini 3 thinking level, and temperature on a compatible model. Changing settings creates a separate saved analysis. Comparing saved reports is descriptive; it is not evidence that one technique is more accurate.

4. Generative AI harness criteria

Requirement	Implementation / demonstration
Standalone application	Browser UI, Django API, relational persistence and background processing.
Real API	Backend-to-Gemini request from Celery; success must be demonstrated with a valid key and live saved response.
Substantive GenAI	Structured priorities, rationales, evidence references, steps and verification proposals—not a chat-only Q&A.
Adjustable parameters	Model, token limit, model-compatible thinking/temperature.
At least four techniques	Zero-shot, few-shot, structured reasoning and role-based prompts.
Error handling	Typed provider failures, bounded retries, schema/evidence validation, queue/stale-job handling.
Documentation and GitHub	This PDF, source documentation, GitHub PRs and current reproducible test evidence.

5. Safety controls and limitations

- Generated output must use an allowed action name and cite supplied C*/F* evidence IDs. Invalid JSON, unrecognized references or unsupported actions are rejected.
- The only implemented data-changing operations in this release are trimming outer whitespace and removing exact duplicate rows, with a deterministic preview.
- Approved transformations create a new dataset with lineage and validation; original CSV bytes are unchanged.
- A duplicate row may be intentional; trimming may change identifiers. A user must review semantic consequences.
- Synthetic data are suitable for demonstrations. Real datasets require data-owner approval and review of sensitive column names and aggregate disclosure.

6. Test evidence and prompt quality evaluation

The repository includes tests for authorization and role isolation, profile and evidence validation, mocked Gemini generation, approved remediation, output checksum and CSV/PDF export. It also includes a read-only report-evaluation command and health probe. Mock tests do not replace a live-provider demonstration.

Recommended controlled comparison: pin one dataset/profile, hold model configuration constant, run all four techniques with multiple repetitions, and manually rate factual grounding, unsafe recommendations, usefulness and acknowledgment of uncertainty. Report exact sample size, failures, latency, and costs rather than only successes.

7. Repeatable verification checklist

- ■ Run docker compose config; build the stack; verify DB/Redis/migration/API/web/worker status.
- ■ Run Django check, makemigrations --check --dry-run, backend pytest and Ruff.
- ■ Run frontend ESLint, TypeScript, Vitest and Next.js production build.
- ■ Compare generated runtime OpenAPI routes with reviewed contracts/openapi.yaml.
- ■ Run scripts/check_repo_secrets.py and scripts/check_release.py against the local API.
- ■ Upload fixtures/customers_quality_challenge.csv and review actual profile values.
- ■ Request Gemini with explicit consent; record a real saved report and its chosen parameters.
- ■ Run each of the four prompt techniques on comparable evidence and document observed outcomes.
- ■ Preview and approve a synthetic remediation, validate immutable source and derived checksums.
- ■ Download derived CSV and PDF audit; verify viewer cannot perform owner/editor actions.
- ■ Attach current GitHub Actions backend/frontend/integration/security check outcomes.

8. Operational readiness and future validation

Before production use, demonstrate secure HTTPS/proxy configuration, secret rotation and storage, isolated persistent volumes, restore-tested backups, cloud health monitoring, vulnerability scans, browser accessibility, real-provider quotas/latency, and measured incident recovery. Local Docker success alone does not establish production security or availability.

9. Submission evidence placeholders

GitHub repository: add repository URL and merged PR references. **CI:** attach latest green job screenshots and recorded test counts. **Live Gemini:** attach a consented synthetic-example screenshot with model/technique, evidence-linked recommendations and failure handling. **Transformation:** include before/after measurements and source/derived SHA-256 comparisons. Do not claim that any blank item has passed.

Technical references in repository: docs/13_EVIDENCE_GROUNDED_AI_REMEDIATION.md; docs/14_CONTROLLED_REMEDIATION_AND_AUDIT.md; docs/15_PRODUCT_HARDENING.md; docs/16_PRODUCT_COMPLETION.md; docs/17_DEPLOYMENT_READINESS.md.